



K.R. Mangalam University

School of Engineering & Technology

Operating System Lab (ENCA252)

Lab Assignment 3

Comprehensive Implementation of Page Replacement
Algorithms

Submitted by:

Name: Bhavya Rattan

Roll Number: 2401201004

Course: BCA (AI & DS)

Submitted To:

Dr. Jyoti Yadav

OS Lab Assignment 3 - Page Replacement Algorithms Report

□ Cover Section

- **Name:** Bhavya Rattan
 - **Roll Number:** 2401201004
 - **Course:** BCA (AI & DS)
 - **University:** K.R. Mangalam University, School of Engineering & Technology
 - **Subject:** Operating System Lab (ENCA252)
 - **Submitted To:** Dr. Jyoti Yadav
 - **Date:** April 25, 2026
-

□ Objective

Implement and analyze five major page replacement algorithms: **FIFO**, **LRU**, **Optimal**, **MRU**, and **Second Chance (Clock)**. The goal is to simulate memory behavior and evaluate performance based on **page faults**.

1. FIFO (First-In, First-Out)

How it works:

The oldest page in memory (the one that arrived first) is replaced when a new page needs to be loaded and all frames are full.

Logic:

A simple queue is maintained. When a page fault occurs and frames are full, the page at the front of the queue (oldest) is evicted and the new page is added to the rear.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☐ No
5	2	3	0	2	☒ Yes
6	1	0	2	1	☒ Yes
7	2	0	2	1	☐ No
8	0	0	2	1	☐ No
9	7	2	1	7	☒ Yes
10	0	1	7	0	☒ Yes
11	1	1	7	0	☐ No
12	2	7	0	2	☒ Yes

Total Page Faults: 8

Code Output:

```
bhavya@dell:~$ cd /home/bhavya/Documents/os/os_assignment_3
bhavya@dell:~/Documents/os/os_assignment_3$ nano page_replacement.py
bhavya@dell:~/Documents/os/os_assignment_3$ python3 page_replacement.py
=====
PAGE REPLACEMENT ALGORITHM SIMULATOR
=====

Enter number of frames: 3
Enter page reference string (space-separated): 7 0 1 2 0 3 0 4 2 3 0 3 2

Reference String : [7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2]
Number of Frames : 3
-----

=====
RUNNING ALL ALGORITHMS
=====

FIFO (First In First Out)
-----
Page | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 |
-----
      | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 0 | 0 | 0 |
      |  | 0 | 0 | 0 | 0 | 3 | 3 | 3 | 2 | 2 | 2 | 2 | 2 |
      |  |  | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 3 | 3 | 3 | 3 |

Total Page Faults : 10
```

2. LRU (Least Recently Used)

How it works:

The page that has not been used for the longest period of time is replaced.

Logic:

Usage history is tracked for each page in memory. On a page fault, the page with the oldest last-use timestamp is evicted. Exploits **temporal locality** — recently used pages are likely to be used again soon.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	0	3	☐ No
5	2	0	3	2	☒ Yes
6	1	3	2	1	☒ Yes
7	2	3	1	2	☐ No
8	0	1	2	0	☒ Yes
9	7	2	0	7	☒ Yes
10	0	2	7	0	☐ No
11	1	7	0	1	☒ Yes
12	2	0	1	2	☒ Yes

Total Page Faults: 9

Code output:

LRU (Least Recently Used)														
Page	7	0	1	2	0	3	0	4	2	3	0	3	2	
	7	7	7	2	2	2	2	4	4	4	0	0	0	
		0	0	0	0	0	0	0	0	3	3	3	3	
			1	1	1	3	3	3	2	2	2	2	2	
Total Page Faults : 9														

3. Optimal Page Replacement

How it works:

Replaces the page that will **not be used for the longest time in the future**.

Logic:

Looks ahead in the reference string to determine which page currently in memory will be needed farthest in the future (or not at all), and evicts that page. This is a theoretical algorithm — it cannot be implemented in real systems as it requires future knowledge.

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☐ No
5	2	7	2	0	☒ Yes
6	1	1	2	0	☒ Yes
7	2	1	2	0	☐ No
8	0	1	2	0	☐ No
9	7	1	7	0	☒ Yes
10	0	1	7	0	☐ No
11	1	1	7	0	☐ No
12	2	2	7	0	☒ Yes

Total Page Faults: 7

Code output:

MRU (Most Recently Used)														
Page	7	0	1	2	0	3	0	4	2	3	0	3	2	
	7	7	7	7	7	7	7	7	7	7	7	7	7	
		0	0	0	0	3	0	4	4	4	4	4	4	
			1	2	2	2	2	2	2	3	0	3	2	
Total Page Faults : 11														

4. MRU (Most Recently Used)

How it works:

The **most recently used** page is replaced when a page fault occurs.

Logic:

Tracks which page was accessed last. On a fault, that most-recently-used page is evicted. This is the opposite of LRU and typically performs poorly for general workloads, but can be efficient for certain access patterns (e.g., sequential scans).

Step-by-Step Trace

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7	-	-	☒ Yes
2	3	7	3	-	☒ Yes
3	0	7	3	0	☒ Yes
4	3	7	3	0	☐ No
5	2	7	2	0	☒ Yes
6	1	7	1	0	☒ Yes
7	2	7	2	0	☒ Yes
8	0	7	2	0	☐ No
9	7	7	2	0	☐ No
10	0	0	2	0	☐ No
11	1	0	2	1	☒ Yes
12	2	0	2	1	☐ No

Total Page Faults: 7

Code output:

MRU (Most Recently Used)														
Page	7	0	1	2	0	3	0	4	2	3	0	3	2	
	7	7	7	7	7	7	7	7	7	7	7	7	7	
		0	0	0	0	3	0	4	4	4	4	4	4	
			1	2	2	2	2	2	2	3	0	3	2	
Total Page Faults : 11														

5. Second Chance (Clock Algorithm)

How it works:

A modified version of FIFO. Each page has a **reference bit**. When a page is accessed, its bit is set to 1. On replacement, if the oldest page's bit is 1, it gets a "second chance" (bit reset to 0, moved to back of queue); if 0, it is replaced.

Logic:

Maintains a circular queue (clock). The "clock hand" sweeps through pages. A page with reference bit = 1 is spared once; on the next sweep if still unreferenced, it is evicted. Approximates LRU with lower overhead.

Step-by-Step Trace (*Bits shown in parentheses: 1 = referenced*)

Step	Page	Frame 1	Frame 2	Frame 3	Page Fault
1	7	7(1)	-	-	☒ Yes
2	3	7(1)	3(1)	-	☒ Yes
3	0	7(1)	3(1)	0(1)	☒ Yes
4	3	7(1)	3(1)	0(1)	☐ No
5	2	2(1)	3(1)	0(1)	☒ Yes
6	1	2(1)	3(1)	1(1)	☒ Yes
7	2	2(1)	3(1)	1(1)	☐ No
8	0	2(1)	0(1)	1(1)	☒ Yes
9	7	2(1)	0(1)	7(1)	☒ Yes
10	0	2(1)	0(1)	7(1)	☐ No
11	1	1(1)	0(1)	7(1)	☒ Yes
12	2	1(1)	0(1)	2(1)	☒ Yes

Total Page Faults: 9

Code Output

```
Second Chance (Clock Algorithm)
Page | 7 | 0 | 1 | 2 | 0 | 3 | 0 | 4 | 2 | 3 | 0 | 3 | 2 |
      | 7 | 7 | 7 | 2 | 2 | 2 | 2 | 4 | 4 | 4 | 4 | 3 | 3 |
      |   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 2 | 2 | 2 | 2 |
      |   |   | 1 | 1 | 1 | 3 | 3 | 3 | 3 | 3 | 0 | 0 | 0 |
Total Page Faults : 9
```

Performance Comparison

```
=====
PERFORMANCE COMPARISON SUMMARY
=====
Algorithm      Page Faults  Efficiency
-----
FIFO           10         23.1%
LRU            9          30.8%
Optimal        7          46.2%
MRU           11          15.4%
Second Chance  9          30.8%
```

Algorithm	Total Page Faults	Efficiency Rank
Optimal	7	1st (Theoretical minimum)
MRU	7	1st (Best real-world for this string)
FIFO	8	3rd
LRU	9	4th
Second Chance	9	4th



Code Output

```
=====
RESULT ANALYSIS
=====
Best Algorithm : Optimal (7 faults)
Worst Algorithm : MRU (11 faults)

Conclusions
-----
• Optimal    → Theoretical minimum faults (not implementable in
               real OSes since future accesses are unknown).
               Used as a benchmark for other algorithms.

• LRU        → Best practical algorithm; exploits temporal
               locality well. Higher overhead due to recency
               tracking.

• FIFO       → Simplest to implement. Can suffer from Belady's
               anomaly (more frames → more faults in rare cases).

• Second
  Chance     → Approximates LRU with lower overhead by using a
               single reference bit per frame (used in Linux).

• MRU        → Useful for sequential scans where recently used
               pages won't be needed again soon (e.g. DB scans).
```

Conclusion

- **Best Performer: Optimal** and **MRU** both achieved 7 page faults — the lowest. Optimal is the theoretical minimum benchmark; MRU matched it coincidentally due to the specific access pattern of this reference string.
- **Worst Performers: LRU** and **Second Chance** both had 9 faults for this input, demonstrating that performance is highly input-dependent and no single algorithm dominates all cases.
- **Observations: FIFO** sits in the middle at 8 faults. Interestingly, **Second Chance** — which is designed to improve on FIFO — performed worse here (9 vs 8), showing that the reference bit mechanism can backfire on short, non-repeating sequences. In production OS design, **LRU approximations** (like Second Chance/Clock) are still the most common choice overall due to their balance of practicality and average-case performance across diverse workloads.